Now we have used the LangChain framework to connect our Vector Database and LLM. We created a RetrievalQA chain with a custom prompt for our chatbot. Last, we tracked and logged our model with MLflow and have it ready for model serving in *Chapter 7*.

Project: Multilabel Image Classification

In this section, we will fine-tune our multilabel classification model. To follow along in your own workspace, please refer to the following notebooks:

- `Ch6-01-Training Base Model`
- `Ch6-02-Scoring Best Model`

Before we begin, note the compute recommendations in the *Technical requirements* section for this project, as you will need a GPU cluster for running the `Ch6-01-Training Base Model` notebook. We would also like to remind you that this example project is focused on code execution, rather than a technical deep dive into domain knowledge. We anticipate that most readers will have no problem running the Image Classification notebooks, but some prior familiarity with the Lightning/PyTorch frameworks will be helpful.

> **Databricks ML Runtimes**
>
> The Databricks Machine Learning Runtimes include pre-built **Deep Learning** (**DL**) infrastructure and libraries like PyTorch, TensorFlow, and Hugging Face transformers, and CUDA libraries. Deep learning models are computationally intensive and distributed deep learning frameworks like **Distributed Data Parallel** (**DDP**) or **Fully Shared Data Parallel** (**FSDP**), DeepSpeed, and Composer are often used for more efficient training. Be sure to check out the Spark PyTorch/DeepSpeed Distributor on Databricks! There is a blog titled "Introducing the Spark PyTorch Distributor" linked in *Further reading* if you are working with PyTorch and you are interested in performing distributed Deep Learning.

Training

We begin our deep learning journey by downloading libraries using `!pip install` in the `Ch6-01-Training Base Model` notebook.

```
!pip install pytorch-lightning==2.1.2 deltalake==0.14.0 deltatorch==0.0.3 evalidate==2.0.
2 pillow==10.1.0
dbutils.library.restartPython()
```

Figure 6.24 – Installing notebook scoped libraries and restarting Python

After installing and loading libraries, we peek at our training data. Then we define our experiment and log paths using some utility functions that you can find in the `mlia_utils` folder.